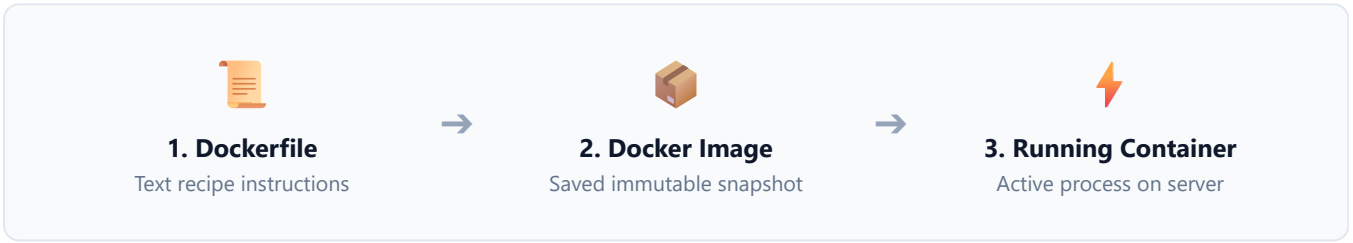
Docker Complete Handbook

From Core Fundamentals to Production Deployment for Spring Boot & MySQL

PROJECT: VRMS-404 | SPRING BOOT 4 • JAVA 21 • MYSQL 8 | PRODUCTION GUIDE

1. Core Concepts Explained Simply

In traditional hosting, apps break because versions of Java, MySQL, or OS libraries differ between development and production. **Docker eliminates this problem** by packaging code, runtime, and configurations together.



Concept	Real-World Analogy	In Your VRMS-404 Project
Dockerfile	A cooking recipe / architectural blueprint	`vrms/Dockerfile` (build steps for Spring Boot)
Docker Image	A packaged frozen meal / Game installer CD	`vrms-404:latest` (frozen package on disk)
Docker Container	Eating the meal / Playing the active game	`VRMS-Backend` (live app running on port 8080)
Docker Compose	Orchestra conductor managing all players	`docker-compose.yml` (runs backend + database)

2. Essential Commands Cheat Sheet

Action	Docker Command	Purpose
Build Image	`docker build -t vrms-404:latest .`	Compiles and packages code
List Images	`docker images`	Shows downloaded/built images
Run Container	`docker run -d -p 8080:8080 vrms-404`	Runs container in background
Check Running Status	`docker ps`	Lists all active containers
View Live Logs	`docker logs -f VRMS-Backend`	Real-time console output
Execute Inside Shell	`docker exec -it VRMS-Backend sh`	Opens terminal inside container
Stop Container	`docker stop VRMS-Backend`	Graceful shutdown
Clean Unused Files	`docker system prune -af`	Reclaims disk space

3. The Production Multi-Stage Dockerfile

A production Dockerfile uses **Multi-Stage Builds**: Stage 1 compiles using full JDK 21, and Stage 2 runs using lightweight JRE 21. This drops image size from ~800MB to under ~200MB and prevents security leaks.

```
# Stage 1: Build & Package (Using JDK 21)
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app

# 1. Copy Gradle wrapper and build configs first (Layer Caching)
COPY gradlew ./
COPY gradle ./gradle
COPY build.gradle.kts settings.gradle.kts ./
RUN chmod +x gradlew

# 2. Copy source code and build executable JAR
COPY src ./src
RUN ./gradlew bootJar --no-daemon -x test

# Stage 2: Minimal Production Runtime (Using JRE 21)
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app

# 3. Copy only the compiled JAR file from the builder stage
COPY --from=builder /app/build/libs/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

> 💡 **Why Layer Caching Matters**
>
> Docker caches each instruction. By copying `gradlew` and `build.gradle.kts` *before* `src`, Docker doesn't re-download dependencies when you change Java source files. Rebuilds take **seconds** instead of minutes.

4. Multi-Container Stack (docker-compose.yml)

VRMS-404 links the Spring Boot backend (`VRMS-Backend`) with MySQL 8.0 (`VRMS-DB`) across an internal virtual bridge network.

```yaml
services:
  # 1. MySQL Database Container
  mysql:
    image: mysql:8.0
    container_name: VRMS-DB
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: vrms
      MYSQL_ROOT_PASSWORD: 12345
    ports:
      - "3307:3306" # Host 3307 avoids conflict with Windows local MySQL
    volumes:
      - mysql_data:/var/lib/mysql
      - ./vrms_database.sql:/docker-entrypoint-initdb.d/01_vrms_database.sql:ro
    healthcheck:
      test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "root", "-p12345"]
      interval: 10s
      timeout: 5s
      retries: 5

  # 2. Spring Boot Application Container
  vrms:
    build:
      context: ./vrms
    image: vrms-404:latest
    container_name: VRMS-Backend
    restart: unless-stopped
    depends_on:
      mysql:
        condition: service_healthy # Waits until MySQL ping answers
    ports:
      - "8080:8080"
    environment:
      SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/vrms?useSSL=false&allowPublicKeyRetrieval=true
      SPRING_DATASOURCE_USERNAME: root
      SPRING_DATASOURCE_PASSWORD: 12345
      SPRING_JPA_HIBERNATE_DDL_AUTO: update

volumes:
  mysql_data:

networks:
  vrms-network:
    driver: bridge
```

> ⚠️ **Why Healthcheck is Crucial**
>
> Without `healthcheck` and `condition: service_healthy`, Spring Boot starts in 1 second while MySQL takes 10 seconds to load. Spring Boot attempts to connect, fails with `Connection Refused`, and crashes immediately. Healthcheck prevents this.

5. Production Cloud Hosting Playbook

Deploying VRMS-404 to a live production server (e.g. DigitalOcean, Hetzner, AWS EC2, Ubuntu 24.04):

Step 1: Install Docker on the Cloud Server

```
# SSH into your VPS
ssh root@YOUR_SERVER_IP

# Install Docker using the official automated script
curl -fsSL https://get.docker.com | sh
docker --version
docker compose version
```

Step 2: Deploy Project Code

```
# Clone your repository
git clone https://github.com/KruyThuna/VRMS-404.git /opt/VRMS-404
cd /opt/VRMS-404

# Start the full stack in detached mode
docker compose up -d --build

# Verify running status
docker compose ps
```

Step 3: Setup Nginx Reverse Proxy with Free HTTPS / SSL

Never expose Spring Boot directly on port 8080 in production. Use Nginx to handle SSL and proxy traffic:

```
# 1. Install Nginx and Certbot
sudo apt install nginx certbot python3-certbot-nginx -y

# 2. Create Nginx Site Config (/etc/nginx/sites-available/vrms.conf)
server {
    server_name api.yourdomain.com;

    location / {
        proxy_pass http://localhost:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}

# 3. Enable site and issue free SSL certificate
sudo ln -s /etc/nginx/sites-available/vrms.conf /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
sudo certbot --nginx -d api.yourdomain.com
```

6. Troubleshooting & Best Practices Checklist

Issue	Symptom	Permanent Solution
Port Bind Conflict	`bind: address already in use`	Change host port in `docker-compose.yml` (e.g. `3307:3306`) so it doesn't clash with local MySQL.
Connection Refused	Spring Boot can't find MySQL	Ensure datasource URL uses container name: `jdbc:mysql://mysql:3306/vrms`, NOT `localhost`.
Data Loss on Restart	Database empty after reboot	Ensure `volumes: - mysql_data:/var/lib/mysql` is configured. Named volumes persist data.
Disk Full	Server runs out of storage	Configure log rotation: `max-size: "10m"`, `max-file: "3"` in `docker-compose.yml`.